Highlights

A Reproducible Edge-to-Cloud Infrastructure for Sustainable Orchestration of Embedded WebAssembly IoT Devices

Luca D'Agati, Giuseppe Tricomi, Fabio Orazio Mirto, Francesco Longo, Giovanni Merlino

- Kubernetes CRDs express intent for constrained non-node IoT devices.
- A TLS/CBOR gateway bridges cloud orchestration and embedded WASM execution.
- Reconciliation policies preserve evidence ownership across cloud, fog, and edge.
- A 100-trial campaign completes every enrollment-to-deployment transaction, with cloud, gateway and device evidence agreeing in each trial and a mean end-to-end orchestration latency of $\approx$1.3 s for a minimal WASM module, inclusive of status-observation granularity.
- Steady-state heartbeat traffic is 1728 B/h per device at the application layer, of order 40 KB/h once TLS and lower-layer framing are included; gateway mediation averages under 5 MiB RAM.

A Reproducible Edge-to-Cloud Infrastructure for Sustainable Orchestration of Embedded WebAssembly IoT Devices

Luca D'Agati[a,b,*], Giuseppe Tricomi[a,b], Fabio Orazio Mirto[a], Francesco Longo[a,b] and Giovanni Merlino[a,b]

[a]Department of Engineering, University of Messina, Messina, Italy
[b]CINI: National Interuniversity Consortium for Informatics, Rome, Italy

ARTICLE INFO

Keywords:
Cloud–Fog–Edge continuum
digital research infrastructures
sustainable orchestration
green experimentation
Internet of Things
WebAssembly
embedded systems
Kubernetes

ABSTRACT

Cloud–Fog–Edge infrastructures increasingly need to deploy and observe application logic across cloud services, gateways, and embedded IoT devices, but conventional cloud-native orchestration assumes Linux-class nodes with container runtimes, resident agents, and rich networking. This leaves a gap for constrained endpoints that must remain lightweight while requiring secure attachment, life-cycle accountability, and reproducible evidence for sustainable research infrastructures. This work addresses the problem of expressing Kubernetes deployment intent for devices that cannot operate as Kubernetes nodes, and translating that intent into verifiable embedded execution. We propose a Kubernetes-native framework that models devices, applications, and gateways through Custom Resource Definitions (CRDs) and uses a gateway-mediated control path to bind device identity, supervise liveness, dispatch WebAssembly (WASM) workloads, and report execution evidence to the control plane. The gateway acts as the semantic boundary between declarative cloud state and constrained device protocols, preserving device simplicity while maintaining status ownership in Kubernetes. The framework is evaluated on a K3s-based test bed with emulated embedded targets across a complete enrollment-to-deployment workflow. Across 100 repeated trials, enrollment, heartbeat supervision, deployment, and transactional completion all succeeded. Heartbeat traffic is 1728 B/h per device at the application layer and of order 40 KB/h on the wire, gateway pod footprint remains in the single-digit MiB and single-digit millicores, and the evaluated device firmware fits within 400 KiB of flash with no kubelet or container runtime. For sustainable research infrastructures, the framework provides a reproducible, measurement-ready substrate for energy-aware placement, instrumentation, and green experimentation policies.

1. Introduction

Digital research infrastructures (RIs), cyber-physical systems, and Internet of Things (IoT) deployments increasingly require software-defined behavior at the edge, where constrained devices execute application logic under reliability, security, timing, life-cycle, and sustainability constraints. Remote deployments can include sensors, actuators, lightweight gateways, and mixed-criticality nodes that must remain inexpensive and portable while still being supervised continuously. In edge-to-cloud RIs, this supervision is not only an operational requirement but also a prerequisite for reproducible experimentation and future energy or carbon accounting across heterogeneous sites. Traditional embedded deployment practices, including manual flashing, board-specific provisioning, and tightly coupled update channels, are workable at small scale but do not provide fleet-wide orchestration or reliable visibility into which software version is running on each device.

In parallel, cloud-native ecosystems have converged around declarative orchestration, reconciliation loops, immutable artifacts, and service-level observability. Kubernetes has become the de facto orchestration substrate for distributed services because it separates desired state from observed state and continuously drives the latter toward the former through controllers [51]. This model has transformed data-center operations, but it also exposes a gap when applied to embedded endpoints. Modern orchestrators assume Linux-capable nodes able to run containers, cluster agents, and rich networking stacks, whereas microcontrollers, sensors, and actuators generally cannot host these components. Nevertheless, cyber-physical deployments still need comparable guarantees for secure attachment, observability, and application life-cycle control.

WebAssembly (WASM) provides a practical execution abstraction for this gap. It supports portable, sandboxed workloads that can run across cloud, fog, and edge environments with a more stable module format than architecture-specific binaries [24]. For embedded systems, WebAssembly Micro Runtime (WAMR) and Zephyr-compatible integrations provide a concrete path toward microcontroller-oriented execution [12, 53]. Runtime feasibility alone, however, does not solve orchestration: a cloud-side intent must still be translated into a secure command for a constrained device, and execution evidence must be reflected back into the control plane.

This paper introduces a full-stack framework that addresses this problem through a gateway-centric architecture for the Cloud–Fog–Edge continuum. The framework builds an end-to-end path from Kubernetes CRDs on K3s, through a Rust/Transport Layer Security (TLS) gateway that mediates identity validation and state propagation, to Zephyr firmware running WAMR on embedded-class targets. In this

*Corresponding author.
Email address: ldagati@unime.it (L. D'Agati)
ORCID(s):

path, the cloud records what should happen, the gateway translates and validates that intent, and the device executes the WASM workload while reporting heartbeat-driven liveness and runtime outcomes. Embedded nodes therefore remain lightweight while still participating in a declarative orchestration model. For research infrastructures, the same path provides stable observation and actuation points that can be reused by sustainability dashboards, energy-aware schedulers, or green experiment-design workflows without moving cloud-native complexity into device firmware.

The framework is not presented as an energy-optimization algorithm, but as a reusable digital-research-infrastructure substrate that makes constrained edge participation observable, repeatable, and ready to be combined with future power, carbon-intensity, and placement policies.

The work is motivated by a concrete engineering observation: secure enrollment is necessary but insufficient. A practical platform must also define ownership of status updates, avoid false disconnection semantics during transient conditions, and expose enough evidence for operators to trust the control plane. This requirement defines what the work integrates: a Kubernetes-native evidence and reconciliation model for non-node embedded WASM endpoints, combining declarative resource modeling, gateway mediation, state reconciliation, and deployment reproducibility into a single verifiable operational path. Separating desired from reported state is the established device-twin pattern (Section 2); the contribution is that the reconciliation point is an ordinary Kubernetes resource rather than a managed service.

1.1. Scope and Contributions

This work presents a systems-oriented contribution at the intersection of IoT orchestration, embedded computing, and secure cyber-physical infrastructure. It integrates established components into a coherent platform that closes an implementation gap frequently acknowledged in the literature but less often documented end to end: how declarative cloud-side intent can be made operational on constrained, non-node embedded devices without sacrificing identity-bound attachment, control-plane visibility, and state fidelity.

The contributions are fourfold:

- a Kubernetes-native resource model with CRDs for devices, gateways, and deployable WASM applications (Section 4);
- a gateway-mediated TLS/Concise Binary Object Representation (CBOR) enrollment and control workflow with explicit identity matching and southbound message translation (Section 5);
- a reconciliation strategy that links desired and reported state while avoiding false disconnect loops and ambiguous status ownership (Section 5);
- an end-to-end K3s validation study with emulated embedded targets and sustainability-oriented observables for wire traffic, firmware footprint, and control-plane resource use, intended as baselines for later green-experimentation studies (Section 7).

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 formulates the problem and design requirements; Section 4 presents the architecture and resource model. Section 5 describes the southbound protocol and reconciliation policy; Section 6 summarizes the implementation stack. Section 7 reports the experimental evaluation. Section 8 discusses implications, validity, and reproducibility. Section 9 concludes the paper.

2. Related Work

This section reviews prior work along the axes that motivate the design in Sections 3–6: edge orchestration, constrained execution, secure IoT communication, reconciliation semantics, and reproducible evaluation. The main gap across these areas is the lack of an end-to-end path from Kubernetes-style declarative intent to verifiable execution on constrained embedded devices that cannot act as ordinary cluster nodes.

2.1. Cloud-Native Edge and IoT Orchestration

Foundational work on fog and edge computing established the need to move computation, storage, and control closer to data sources in order to reduce latency, improve locality, and support cyber-physical applications [8, 14, 43, 47]. Kubernetes has become the dominant orchestration substrate for cloud-native services because it provides declarative resource management, controllers, reconciliation loops, and extensibility through Custom Resource Definitions (CRDs) and operators [28, 51]. Lightweight distributions such as K3s and MicroK8s reduce deployment overhead and make cluster-style operation feasible at smaller sites [13, 38].

Several systems extend Kubernetes semantics toward the edge. KubeEdge moves parts of Kubernetes control closer to edge nodes, OpenYurt targets cloud-edge application management, Akri discovers and exposes leaf devices to Kubernetes, and Krustlet explored WebAssembly workloads as Kubernetes-scheduled units [2, 26, 27, 36]. Recent Kubernetes-native IoT orchestration based on Stack4Things (S4T), CRDs, and Crossplane abstractions similarly demonstrates the value of declarative resource management for IoT deployments [17]. That line has since been extended toward a deviceless Function-as-a-Service model, in which users deploy and invoke functions according to capability, location, and load without managing the underlying devices, using Nuclio as the serverless runtime on edge workers and WebSocket tunnelling to reach nodes behind NATs or firewalls [31]. That design shares the goal of removing device-level management from the user, but its edge workers are Linux single-board hosts running a container-based function runtime, and the tunnelled node remains the unit of execution; the endpoint considered here executes a WASM module under Zephyr/WAMR with no operating-system userspace, container runtime, or cluster agent, so the mediation boundary sits at the gateway rather than at the

device itself. These approaches motivate the present work, but they generally assume Linux-class edge hosts, gateway nodes, container-compatible execution environments, device plugins, or platform services attached to capable nodes. The target considered here is narrower and more constrained: an embedded endpoint that cannot run a kubelet, container runtime, Kubernetes agent, or full cloud-native network stack.

IoT middleware and industrial edge platforms provide additional context. EdgeX Foundry, FIWARE, oneM2M, OMA Lightweight M2M, and OPC UA address aspects of device modeling, interoperability, telemetry, and industrial communication [19, 23, 33–35]. These frameworks are broader than a single orchestration mechanism and often emphasize interoperability or application enablement rather than Kubernetes-native desired-state reconciliation. The contribution is therefore not another general IoT middleware layer, but a focused integration that links cloud-style intent, gateway-mediated secure control, and embedded WASM execution through explicit resource ownership.

The device twin, or shadow, pattern addresses the same problem from the platform side: a cloud-resident document holds desired and reported state for a device that cannot be queried directly, and a managed service reconciles the two as the device reconnects. AWS IoT Greengrass [3], Azure IoT Edge [30] and the LwM2M object model [35] have long provided this. The evidence model used here is the same pattern; what differs is the location of the reconciliation point. In twin-based platforms authoritative state lives in a proprietary service and reaches cluster tooling through bridges. Here it is a Kubernetes resource from the outset, so status ownership, admission control, watches and the controller pattern extend to constrained devices directly, and a placement policy is an ordinary controller over the same objects.

2.2. Constrained Execution and Secure Communication

Embedded and IoT operating systems provide the substrate on which constrained devices communicate and execute application logic. Zephyr, FreeRTOS, RIOT, and Contiki-NG represent different points in the design space for real-time behavior, connectivity, portability, and resource footprint [4, 6, 18, 53]; TinyOS established the long-standing importance of event-driven and resource-aware design in sensor-network systems [29]. These operating systems make embedded applications feasible, but they do not by themselves provide a cloud-side mechanism for expressing intent, binding identity, reconciling state, and verifying deployment outcomes.

WebAssembly adds a portable execution layer that can complement embedded operating systems. The original WebAssembly design emphasized safe, compact, and efficient portable code [24, 57], while the WebAssembly System Interface (WASI) extended the ecosystem toward non-browser execution [11]. Runtime implementations such as WAMR, Wasmtime, WasmEdge, and wasm3 demonstrate that WASM can execute outside browsers and, in some cases, in resource-constrained settings [10, 12, 55, 56]. Container and cloud ecosystems have also begun to integrate WASM through projects such as runwasi and Spin [16, 22]. These efforts establish runtime feasibility, but not operational orchestration: the unresolved question is how a cloud control plane can safely decide that a device should run a given module, transmit that intent through a trusted gateway, and receive evidence that execution and liveness match the requested state.

Secure IoT communication relies on standardized transport, object security, and compact serialization mechanisms. TLS 1.3 and DTLS 1.3 provide channel security [39, 40]; CoAP and MQTT address lightweight application-layer communication [32, 46]; OSCORE and COSE provide object security and compact cryptographic containers [44, 45]; and CBOR supports low-overhead binary serialization [9]. These mechanisms are complementary to the present work. A secure channel can protect messages in transit, but it does not determine which component owns reported state, when a device should be considered connected, or how a cloud resource should converge after a deployment acknowledgment.

2.3. Reconciliation, Evaluation, and Positioning

The operator pattern and Kubernetes controller model encode desired state, observe actual state, and repeatedly reconcile the two [28, 51]. In conventional clusters, this model works because nodes and workloads participate directly in the Kubernetes ecosystem. Edge systems complicate the model because connectivity can be intermittent, observations may be delayed, and devices may expose limited introspection. Prior platforms address parts of this challenge through edge agents, device twins, or local gateways [2, 19, 27, 35, 36]. For embedded targets, however, the gateway becomes more than a message forwarder: it is the semantic boundary between cloud resources and protocol-derived evidence.

Reproducible evaluation is equally important because physical hardware availability, network conditions, and manual setup can make embedded IoT experiments difficult to repeat. Large-scale experimental infrastructures such as SLICES highlight the role of programmable, shared scientific instruments for networking research [21]. Renode supports hardware-faithful emulation and multi-node embedded scenarios, while Cooja, ns-3, and OMNeT++ support repeatable simulation and network experimentation in wireless, sensor, and IoT research [5, 37, 42, 54]. The evaluation in this paper follows that direction by using an emulation-driven validation path and structured measurement records. This is appropriate because the main claim concerns end-to-end orchestration coherence rather than radio performance, energy consumption, or microarchitectural timing.

The framework connects these strands at a narrower integration point than the middleware-centric approaches above: its main abstraction is not a device model or a telemetry

Table 1
Qualitative comparison relative to representative edge, IoT, and WASM orchestration approaches.

Approach	Primary abstraction	Assumed edge capability	Kubernetes integration	Gap for this work
KubeEdge/OpenYurt	Cloud-edge node and application management	Capable edge hosts or edge nodes with local agents	Extends Kubernetes control toward the edge	Too heavy for endpoints that cannot host cluster agents.
Akri/device-plugin style integration	Discovery and exposure of leaf devices	Kubernetes nodes near the device	Represents devices through cluster-attached resources	Does not make the constrained device itself a verifiable WASM execution target.
Krustlet/runwasi-style WASM execution	WASM workload as schedulable unit	Host able to participate in Kubernetes scheduling	Integrates WASM into Kubernetes workload execution	Targets worker-like execution environments rather than non-node embedded firmware.
Edge FaaS/deviceless serverless	Function as deployment unit, with capability- and load-aware placement	Linux single-board edge workers able to host a container-based function runtime	Kubernetes- and Docker-based deployment of the function runtime	Assumes an edge worker that can run a container runtime, not a non-node embedded endpoint.
Device twin/shadow platforms	Cloud-resident desired/reported state document per device	Any device able to reach the cloud endpoint	External to Kubernetes; bridged after the fact	Reconciliation lives in a managed service rather than in cluster-native resources.
IoT middleware	Device model, telemetry, interoperability, and management	Gateway, server, or device-management infrastructure	May be bridged to cloud platforms but is not primarily a CRD reconciliation model	Provides broad IoT management, but not Kubernetes-native deployment intent to embedded WASM execution evidence.
This framework	CRD intent plus gateway-mediated evidence	Zephyr/WAMR endpoint with TLS/CBOR protocol support only	Uses CRDs, controllers, and status reconciliation without making the device a worker node	Focused on reproducible orchestration of constrained embedded WASM targets.

bus, but the convergence between CRD intent, gateway evidence, and embedded WASM execution. The contribution is therefore best understood as a systems-integration study that combines explicit identity binding, gateway-mediated control, heartbeat-based liveness, CRD status convergence, and reproducible quantitative validation.

Table 1 summarizes frameworks' characteristics. The comparison is qualitative because the surveyed systems target different layers of the edge continuum, but it clarifies the specific gap addressed here: preserving Kubernetes-native intent and reconciliation while keeping the embedded endpoint outside the cluster and still obtaining execution evidence.

3. Problem Formulation and Design Requirements

The platform addresses the following problem:

> *How can a Kubernetes-based control plane reliably express the intent that a constrained device should execute a specific WebAssembly module, route that intent through a secure gateway, and verify the resulting execution and liveness state, with measurable control-plane and southbound overhead compatible with sustainable research-infrastructure operation, without requiring the device to behave like a Kubernetes node?*

Cloud orchestrators normally assume Linux-capable workers with container runtimes, cluster agents, and rich networking support. Embedded devices instead provide limited compute, memory, and network functionality, but they still need comparable guarantees for identity, observability, and life-cycle accountability. The framework therefore treats orchestration as an end-to-end translation problem between cloud intent, gateway-mediated protocol control, and edge-side WASM execution.

3.1. Design Pillars

The problem is structured around three pillars, corresponding to the three levels at which the framework acts:

- At the resource-model level, Kubernetes assumes a declarative control plane with explicit status ownership, whereas embedded deployments often rely on imperative, device-local procedures that are opaque to the cloud.
- At the transport level, constrained devices need lightweight and secure communication paths, but the cloud side requires rich enough semantics to express enrollment, liveness, and application control.

Table 2
Traceability between design requirements, architectural decisions, and evaluation observables.

Requirement	Rationale	Design consequence	Evaluation observable
R1: Declarative intent	Operators need a durable statement of what should run even when a device is offline or temporarily unreachable.	Device, Application, and Gateway state is represented through Kubernetes CRDs rather than through ephemeral gateway memory alone.	Application desired state, Device phase, Gateway association, and status convergence.
R2: Identity binding	A protected channel is insufficient unless the endpoint is bound to the expected platform identity.	The gateway validates enrollment material before associating a TLS session with a Device resource.	Enrollment success, expected identity match, and gateway runtime view.
R3: Lightweight edge execution	Constrained endpoints cannot host kubelet, container runtimes, or Linux userspace services.	The device firmware executes portable WASM modules through WAMR on Zephyr.	Firmware footprint, WASM deployment result, and absence of edge cluster agents.
R4: Evidence-driven status	Control-plane state must describe observed execution rather than only command dispatch.	Gateway-propagated acknowledgments and results are treated as status evidence.	Deployment phase, execution outcome, heartbeat freshness, and transactional success count.
R5: Conservative liveness	Intermittent visibility must not immediately erase the latest valid attachment state.	Controllers avoid interpreting a single stale or incomplete observation as definitive disconnection.	Heartbeat observability, connectivity state, and cross-layer evidence checks.
R6: Reproducible sustainability baseline	Later energy and carbon policies require a repeatable orchestration substrate with known overhead.	The testbed records wire size, firmware footprint, and pod-level resource samples.	Bytes per hour per device, firmware flash/RAM footprint, gateway RAM, and gateway CPU.

- Operationally, even when protocol messages are correct, reconciliation logic must interpret transient conditions conservatively to avoid reporting an incorrect device state.

3.2. Design Requirements

These pillars lead to a compact set of requirements. Desired and observed state for devices, applications, and gateways must remain represented in Kubernetes resources. Enrollment must occur over a protected channel, with an auditable binding between the identity material a device presents and the platform-level resource identity expected for it. The application life cycle must use a format usable across cloud, fog, and constrained edge targets, motivating WebAssembly as a common execution abstraction. Runtime evidence must confirm enrollment, heartbeat-based liveness, deployment status, and gateway association without manual inspection of device internals. For digital research infrastructures, the same evidence model should expose where workload placement, liveness, and execution outcomes can later be correlated with site-level power or carbon signals. Finally, the complete stack must deploy and test in a commodity research environment.

Table 2 makes these requirements explicit and links each one to the architectural decision and evaluation observable used later in the paper.

Communication, orchestration, and status management are therefore co-designed rather than layered opportunistically. Embedded devices are not trusted to manage cluster state directly, and Kubernetes is not expected to maintain device sessions directly: a managed gateway acts as the trusted fog-layer mediation point while the control plane remains authoritative for desired state.

4. Architecture

4.1. Layered View

The framework is organized around the Cloud–Fog–Edge continuum. The **cloud layer** hosts the API server, dashboard, controllers, CRDs, and supporting services on Kubernetes; it stores desired state and persists reported execution state. The **fog layer** hosts the Rust-based gateway and emulation support services; it translates orchestration events into southbound TLS/CBOR messages, validates device identity, and propagates status updates back to the cloud. The **edge layer** contains Zephyr firmware, the WAMR execution environment, and deployed WASM modules; it executes the intended behavior and emits heartbeat and runtime evidence.

This decomposition localizes complexity where it is most affordable: Kubernetes retains declarative control, the gateway absorbs protocol and identity mediation, and the device remains lightweight. Figure 1 summarizes this path: declarative intent flows from the cloud toward the edge, while runtime evidence returns through the gateway into Kubernetes status fields.

4.2. Gateway-Centric Control

Gateway-centric mediation is the main architectural choice. Devices do not manipulate Kubernetes resources or embed cluster-specific logic. Instead, the gateway terminates southbound TLS sessions, validates enrollment messages, binds transport sessions to device identities, dispatches application life-cycle commands, and reports execution and liveness evidence to the cloud side.

This design keeps constrained devices small while concentrating protocol policy, identity checks, message normalization, and status propagation at a controllable boundary. It also gives the cloud side a stable integration surface

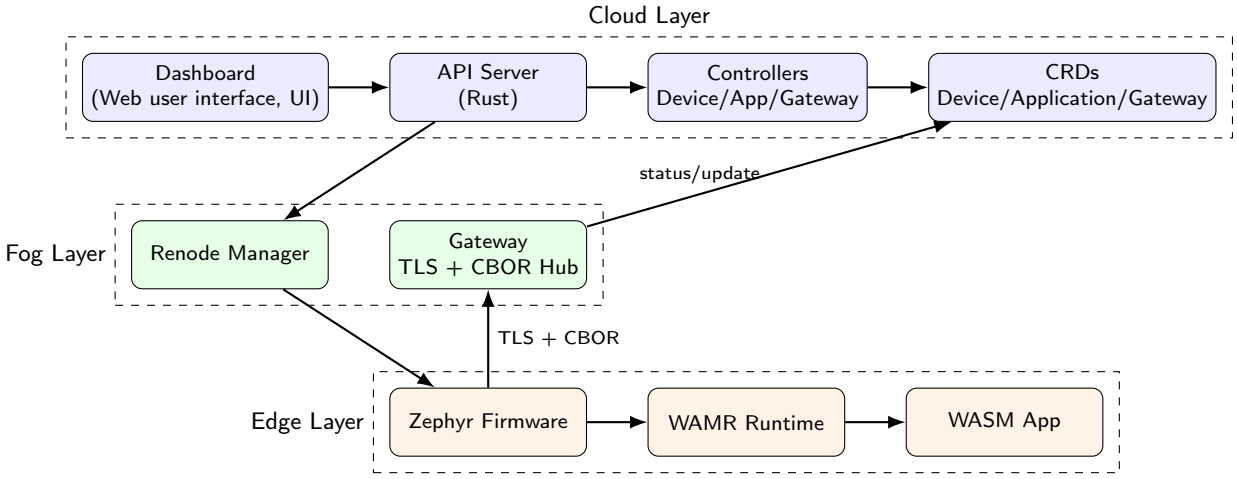

Figure 1: High-level architecture of the proposed framework. Declarative cloud intent is mediated by the gateway and translated into verifiable embedded WASM execution.

Table 3
Primary CRDs and control/evidence ownership.

Resource	Desired or configured state	Reported evidence
`Device`	Platform identity, expected key, gateway preference, runtime attributes	Gateway association, phase, connectivity, heartbeat freshness
`Application`	Target device, WASM artifact, desired execution state	Deployment phase, start outcome, execution result or error evidence
`Gateway`	Endpoint metadata, capabilities, namespace-scoped mediation path	Availability metadata, resolved devices, status propagation context

if the device protocol evolves. Gateway correctness and availability are therefore explicit managed properties of the fog layer, rather than implicit assumptions distributed across constrained endpoints.

4.3. Resource Model

The framework relies on three primary CRDs, summarized in Table 3. The `Device` resource stores identity, gateway preference, runtime attributes, expected public key, and reported connectivity state. The `Application` resource expresses the desired WASM deployment for a target device and records execution outcomes in status fields. The `Gateway` resource describes endpoint metadata, capabilities, and the information needed to resolve the mediation path.

The resource model separates desired state from reported state. This preserves deployment intent when devices are offline, unreachable, or in transition, and it helps operators compare requested state with the latest evidence observed by the gateway and controllers.

4.4. Control and Observability Boundaries

The architecture distinguishes control ownership from evidence ownership. Operators and higher-level services change desired state through Kubernetes resources; the gateway turns protocol-level observations into platform-level status updates; and controllers enforce reconciliation and consistency. This separation reduces ambiguous responsibility and limits conflicting status writes. The validation campaign specifically exercises this boundary by checking that transient visibility gaps do not overwrite fresh gateway evidence or produce false disconnection state.

5. Protocol and Reconciliation

5.1. Southbound Life-Cycle

The southbound protocol uses framed CBOR messages over TLS. It is compact enough for embedded-class clients, but still supports identity binding, acknowledgments, liveness evidence, and application deployment. The life-cycle has three phases: enrollment, heartbeat-based supervision, and resource-driven WASM deployment.

Enrollment establishes the transition from a protected transport session to a platform-recognized device identity:

1. The device opens a protected session with the gateway and requests enrollment.
2. The gateway validates the request before proceeding.
3. The device provides identity material that can be bound to a known platform resource, and signs a single-use gateway nonce to demonstrate that it holds the matching private key.
4. The gateway verifies the signature and compares the material with the identity registered in the control plane.
5. The device confirms that enrollment information has been received and that it is ready for managed operation.
6. The gateway stores the device–session association and updates the corresponding resource status.

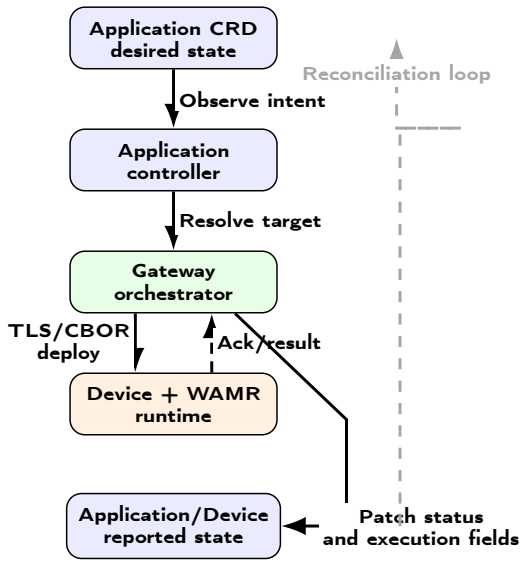

Figure 2: Application deployment workflow and status-reconciliation loop.

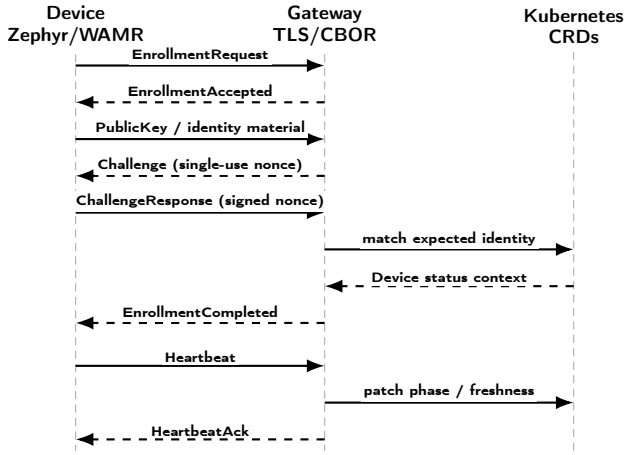

Figure 3: Enrollment and heartbeat sequence. The device proves key possession by signing a single-use nonce before its evidence is reflected into Kubernetes status.

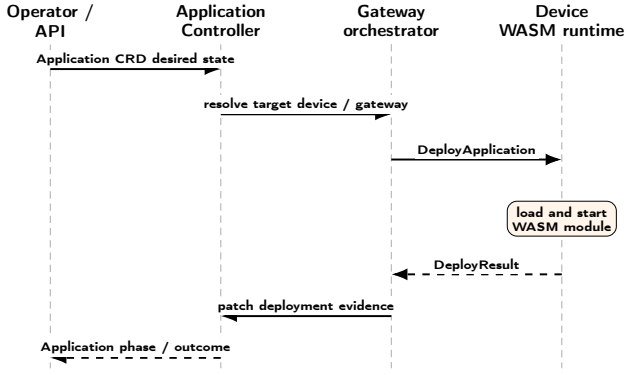

Figure 4: WASM deployment sequence. Declarative application intent is translated by the gateway into device execution and reconciled as application status.

After enrollment, periodic heartbeats maintain liveness visibility. At the communication layer, they show that the protected channel and device remain responsive. At the control-plane layer, they provide evidence used to refresh reported connectivity and device phase. Missing or delayed observations are interpreted conservatively, so transient gaps do not immediately become disconnection events.

WASM deployment follows the same evidence-driven pattern but starts from declarative intent: an operator declares that a target device should run a workload, the control plane resolves the mediation path, the gateway translates the intent into protected messages, the device loads the module through its embedded runtime and returns execution evidence, and the gateway propagates the outcome into control-plane status (Figure 4).

Figure 2 summarizes the deployment loop: the controller resolves intent, the gateway mediates the southbound command, the device reports an outcome, and the control plane reconciles status. This preserves the declarative pattern and remains compatible with retry, staged rollout, and multi-gateway scheduling policies.

5.2. Protocol Sequences

Figures 3 and 4 summarize the southbound protocol interactions evaluated in Section 7. The diagrams emphasize role ownership: the device initiates attachment and emits runtime evidence, the gateway validates and translates protocol events, and the Kubernetes control plane stores desired and reported state.

The decisive event in enrollment is not the opening of a protected session but the association between the device-provided identity material and the expected `Device` resource, admitted only once possession of the matching private key is proved; heartbeats then refresh that association. Deployment runs the complementary direction, and the returned `DeployResult` is treated as execution evidence rather than as a transport acknowledgment: application status converges only once that evidence is propagated.

5.3. Reconciliation Policy

Protocol correctness becomes platform correctness only when message outcomes are interpreted consistently by the control plane. The framework therefore gives the gateway authority over protocol-derived observations and constrains controllers to reconcile those observations without overreacting to transient visibility gaps.

Integration testing exercised short-lived differences between gateway observations and control-plane status, confirming that liveness should not be inferred from a single incomplete observation. The resulting policy preserves the latest valid gateway association, avoids immediate disconnection when heartbeat evidence and transport visibility briefly diverge, and waits for a subsequent reconciliation cycle before treating the condition as a real state transition. Reconciliation policy is therefore part of the architecture rather than an incidental implementation detail.

Four constants govern the policy. The firmware emits a heartbeat every 25 s; the gateway treats an association as

Table 4
Implementation-facing conventions for the validated gateway-mediated control path.

Element	Minimum evidence carried or recorded	Status/reconciliation use
Enrollment	Device identifier, firmware/runtime descriptor, device public-key material or fingerprint, gateway identifier, and freshness context for the protected session	The gateway binds the transport session to the expected `Device` resource before reporting connectivity or phase updates.
Heartbeat	Device identifier, gateway/session association, receive timestamp, and heartbeat freshness window	Controllers interpret heartbeat evidence conservatively: stale or missing observations do not overwrite the latest valid association until a subsequent reconciliation cycle confirms a state transition.
Deployment	Application identifier, target device, WASM artifact reference or payload hash, desired execution state, result status, and error or exit evidence when available	`Application` status converges only after gateway-propagated execution evidence; desired state remains intact when deployment evidence is missing, stale, or inconsistent.
Measurement record	Trial identifier, enrollment result, heartbeat-freshness check, deployment result, gateway runtime view, `Device` status, `Application` status, timeout outcome, and latency timestamps	A trial is counted as transactionally successful only when all evidence views agree; otherwise the trial is recorded as failed rather than inferred from a single eventually consistent field.

stale after 90 s without one, evaluated on a 30 s sweep; Device and Application controllers requeue every 30 s. The staleness threshold is 3.6× the emission period, so at least three consecutive heartbeats must be lost before a device leaves `Connected` for `Unreachable`, detected with ±30 s granularity. Oscillation is precluded by construction rather than by damping: the timeout transition is one-directional and only a fresh heartbeat restores `Connected`, so a single delayed observation cannot produce a disconnect–reconnect cycle.

For reproducibility, the protocol messages evaluated here should therefore be read as typed CBOR records with explicit identity, freshness, target, artifact, and outcome fields, even when the diagrams show only message names. The implementation-facing invariant is that a transport acknowledgment alone is insufficient for success: Kubernetes status is updated only after the gateway has matched the device identity, preserved the gateway association, and propagated fresh deployment or liveness evidence.

6. Implementation

The implementation combines Rust control-plane services with Zephyr-based device firmware and WAMR execution. The API server, gateway, and controllers are implemented in Rust, using its memory-safety guarantees and asynchronous networking support. Resource interaction follows Kubernetes controller and status-patching practices [28, 51], allowing embedded endpoints to be managed as first-class resources without treating them as cluster nodes. On the device side, Zephyr provides the embedded networking and TLS substrate [53], while WAMR provides lightweight WASM execution on constrained targets [12].

The reference deployment runs on K3s [38]. In the presented framework, K3s denotes the concrete lightweight Kubernetes distribution used for the experiments, whereas K8s/Kubernetes denotes the generic API, manifests, CRDs, and controller model implemented by the framework and reflected in the repository naming. It uses namespace-scoped resources and locally built images, preserving real Kubernetes API and reconciliation semantics while remaining accessible on a single-node Linux environment for development, experimentation, and repeatable evaluation. The deployment includes the API services, dashboard, gateway/controller components, CRDs, and supporting services needed to exercise enrollment, deployment, and status propagation.

Responsibilities are distributed across specialized controllers rather than concentrated in a single service. Device-oriented reconciliation manages attachment state and gateway resolution. Application-oriented reconciliation interprets deployment intent and maps it to gateway actions. Gateway-oriented reconciliation maintains metadata needed for path resolution and status interpretation. This decomposition follows the CRD model and reduces the risk that unrelated concerns become coupled inside one control loop.

6.1. Control-Plane Components

The control-plane implementation is organized around a narrow API layer and a set of reconciliation workers. The API server is responsible for user-facing operations, validation of high-level requests, and creation or update of Kubernetes resources. It does not maintain authoritative device-session state. That responsibility is delegated to the gateway and reflected through CRD status fields, so that the Kubernetes API remains the persistent source of desired state and the gateway remains the source of southbound observations. This split prevents the dashboard or API server from becoming a hidden second control plane.

The Device controller interprets gateway-reported attachment evidence rather than opening connections itself, maintaining the selected gateway, the connectivity phase, heartbeat freshness and the last valid device–session association. The Application controller turns a desired deployment into a gateway action once target and mediation path resolve, and the Gateway controller keeps the mapping

Table 5
Implementation responsibilities across the validated stack.

Component	Main responsibility	State written or observed	Failure containment
Dashboard and API server	Accept operator requests, validate high-level parameters, and create or update CRD objects.	Application desired state, Device metadata, and Gateway references.	No direct southbound session ownership.
Device controller	Interpret attachment, phase, heartbeat freshness, and gateway association.	Device status fields and gateway-derived evidence.	Avoids replacing valid status from one transient missing observation.
Application controller	Resolve target device and gateway, dispatch deployment intent, and reconcile execution result.	Application status, deployment phase, and outcome evidence.	Keeps desired state intact if execution evidence is absent or inconsistent.
Gateway runtime	Terminate TLS sessions, decode CBOR frames, bind device identities, and forward deployment commands.	Runtime session map and protocol-derived status patches.	Isolates protocol errors from Kubernetes controllers and firmware.
Zephyr/WAMR firmware	Maintain the protected channel, emit liveness evidence, load the WASM module, and report execution result.	Heartbeat messages, deployment acknowledgments, and runtime result fields.	Does not write Kubernetes resources or store cluster credentials.

between cloud resources and fog endpoints explicit. Table 5 summarizes these responsibilities.

6.2. Gateway Runtime State

The gateway runtime maintains the small amount of mutable state that cannot naturally live inside Kubernetes alone: active TLS sessions, decoded device identifiers, freshness information for recently received heartbeats, and pending deployment operations. This state is intentionally treated as a cache of protocol reality rather than as a replacement for Kubernetes status. When a device enrolls, the gateway creates a session-to-device binding only after the enrollment material is compatible with the expected Device resource. When a heartbeat arrives, the gateway refreshes liveness evidence and exposes the new observation to the control plane. When an Application deployment is requested, the gateway checks that the target device is currently reachable through a known session before emitting the southbound command.

This organization gives the framework a precise failure boundary. If the firmware disconnects, the gateway loses the active transport session but the Device resource still retains the latest desired configuration and the latest valid reported association until reconciliation determines that the condition is stale. If the Kubernetes API is temporarily unavailable, the gateway can still preserve the immediate transport view, but durable convergence waits until status patches are accepted. If the gateway restarts, the cloud-side desired state survives and devices must re-establish session-bound evidence. These cases are not hidden behind a best-effort transport acknowledgment; they are reflected as missing, stale, or inconsistent evidence that controllers can interpret conservatively.

The southbound channel is mutually authenticated: each device holds an ECDSA P-256 key pair and a certificate issued by a fleet authority, the gateway requires that certificate during the handshake, and the firmware verifies the gateway against the same authority. Identity binding rests on two checks rather than on presenting an identifier: the key announced at enrollment must be the one the peer authenticated the transport with, and the device must sign a single-use gateway nonce with the matching private key before any association is recorded. A public key is not a secret, so the second check separates holding an identity from having observed one, and the nonce is consumed whether or not the signature verifies, so a captured response cannot be replayed. Dispatched modules carry a SHA-256 digest the firmware recomputes before instantiation. Three limits remain: TLS 1.2 rather than 1.3; one authority with no revocation path; and a digest that binds a module to what the gateway announced, not to a signature by its publisher. Workload identity attestation [48] and signed artifact metadata [41, 52] close these gaps. On the emulated STM32F746G target the signature costs 79 ms and enrollment completes 86–182 ms after the TLS session opens.

6.3. Embedded Execution Path

On the embedded side, the firmware implements only the southbound protocol roles required by the orchestration model. It establishes the protected channel, sends enrollment and heartbeat records, receives deployment commands, passes the WASM payload to WAMR, and returns a structured result. It does not embed Kubernetes client logic, service-account credentials, resource watches, or container-image management. The edge endpoint is therefore closer to a managed execution target than to a miniature cluster node. This is a deliberate implementation constraint: the more Kubernetes behavior is pushed into the firmware, the harder it becomes to keep the device portable, auditable, and suitable for constrained deployments.

The WASM payload used in the evaluation is intentionally minimal so that the measurement focuses on orchestration overhead rather than application computation. Larger WASM modules, richer WASI support, and application-specific input/output channels sit above the same deployment and evidence path and leave the control ownership

Table 6
Experimental testbed and software configuration.

Item	Configuration
Host OS	Ubuntu 24.04 LTS (kernel 6.8.0), x86_64 KVM guest
Host resources	32 vCPUs, 62 GiB RAM, 293 GiB SSD
Orchestrator	K3s v1.35.4+k3s1, single control-plane node
Southbound emulation	Renode `antmicro/renode:nightly`, board `Stm32F746gDisco`
Firmware stack	Zephyr RTOS 3.5.0, WAMR 2.4.1, minimal WASM test module (33 B)
Device networking	Bridge `wasmbr0` at 192.168.1.1/24, one TAP per device, DHCP, DNAT to the gateway pod's TLS port

model unchanged. Their feasibility on this target is a separate question, and a tight one: against the 327,680 B of internal SRAM on the STM32F746NG, the 270,238 B static allocation reported in Section 7 leaves roughly 56 KiB for the WAMR heap, the stacks and the module itself. Establishing the largest module the path can carry, and whether the transfer needs chunking and an integrity check, requires measurement not attempted here.

7. Experimental Evaluation

7.1. Evaluation Design and Testbed

The evaluation combines (i) a functional validation campaign over the full enrollment-to-deployment path and (ii) a sustainability-oriented measurement pass on the same testbed. The functional study asks whether cloud intent, gateway mediation, and embedded WASM execution produce mutually consistent evidence under repeatable conditions. The sustainability pass quantifies southbound wire sizes, edge firmware footprint, and Kubernetes pod resource use.

Experiments run on a single-node K3s cluster with API server, gateway, application controller, and CRDs active. The southbound endpoint is a Renode-emulated STM32F746 running Zephyr firmware with WAMR and connects over TAP-based networking with DNAT to the gateway TLS port. Table 6 summarizes the host and software stack.

After one warm-up run, the campaign executes n=100 repeated enrollment-to-deployment trials started from a known logical state. Each trial performs enrollment verification, heartbeat supervision, and deployment of the minimal WASM module to the same emulated device. A trial is *transactionally* successful only if all three stages succeed and deployment evidence is consistent across the gateway runtime view and the Application CRD (`phase=Running`, device connected, fresh heartbeat). Continuous latencies use monotonic client-side timers; we report mean, standard deviation, median, p95, p99, IQR, CV, and 95% CI of the mean ($\bar{x} \pm t_{0.975,n-1}\, s/\sqrt{n}$, with $t_{0.975,99}$=1.984). For binary outcomes we use Wilson score intervals; with 100/100 successes the 95% lower bound is 96.3%.

Consecutive trials share a host, a warmed cache and long-lived control-plane processes, so they are repetitions rather than independent draws, and the t-based intervals summarize dispersion within this campaign rather than across deployments. The series is stationary after the discarded warm-up run: lag-1 autocorrelation −0.09, Spearman correlation with trial index −0.14 (t=−1.42), and half-campaign means of 1212.1 ms and 1159.5 ms (Figure 6). The campaign also induces no faults, so 100/100 establishes that the evidence-convergence criterion of Table 7 holds whenever the mechanism runs without induced failure, not an availability figure for a fleet.

7.2. Trial Procedure and Evidence Semantics

The harness starts from a known logical state, observes the device enrollment path, waits for heartbeat evidence to become visible through the gateway, creates or updates the Application intent, and then checks whether the gateway runtime view and Kubernetes status converge to the same deployment outcome.

The record separates protocol evidence from the TLS/C-BOR channel, gateway evidence from the runtime session association, Kubernetes evidence from Device and Application status after reconciliation, and timing evidence from the harness's monotonic timestamps. A trial succeeds only when all four agree before the configured timeout, so success cannot be reported from a single eventually consistent field.

Latency measurements follow the same separation. Enrollment latency measures the interval needed to complete the identity-binding path and expose the enrolled device to the control plane. Deployment latency measures the path from declared Application intent to reported execution outcome. End-to-end latency is the sum of enrollment and deployment for the evaluated transaction. Heartbeat observability is reported separately because it measures how quickly the harness can observe a fresh heartbeat after it is available through the gateway; it is not the firmware heartbeat period and should not be interpreted as a liveness interval.

7.3. Functional Validation Results

All 100 transactional trials completed successfully: enrollment, heartbeat supervision, deployment, and cross-layer evidence checks never failed. These trials exercise the enrollment path in which the Device resource reaches `phase = Connected` from gateway-observed session evidence, which is the path described in Section 5 and the one the framework uses in normal operation. Section 7.4.5 reports a later pass in which an explicit board registration precedes the same enrollment path, placing resource creation inside the measured window. Every measured stage succeeded in all trials, so the meaningful variation lies in latency and its distribution.

Table 8 reports the latency profile. Mean end-to-end latency (enrollment plus deployment; heartbeat observability

Table 7
Trial-level success and failure interpretation.

Condition	Trial interpretation
Enrollment accepted but identity not matched to the expected Device resource	Failure, because a protected channel alone is not a platform-level attachment.
Heartbeat received by the gateway but not reflected as fresh liveness evidence	Failure for the transaction, because the control plane cannot rely on the observation.
Deploy command acknowledged but Application status does not converge	Failure, because command dispatch is not equivalent to verifiable execution.
Application status reports success while gateway or Device evidence is inconsistent	Failure, because cross-layer evidence does not agree.
Enrollment, heartbeat freshness, gateway association, and Application outcome agree	Transactional success.

Table 8
Latency profile (n=100 trials). All values in milliseconds.

Stage	Mean	Median	p95	p99	IQR	CV	95% CI of mean
Enrollment	609.5	568.1	839.6	889.8	50.4	0.164	[589.6, 629.4]
Heartbeat obs.	4.08	4.01	4.97	5.29	0.53	0.114	[3.99, 4.17]
Deployment	676.5	574.1	1021.8	1145.2	228.4	0.428	[619.1, 733.9]
End-to-end	1286.0	1168.7	1661.5	1974.3	270.7	0.234	[1226.4, 1345.6]

reported separately) is 1286.0 ms with 95% CI [1226.4, 1345.6] ms. Deployment latency covers the synchronous deployment request followed by a status query, and the harness retries that query on a two-second cycle when the first one does not yet see a terminal phase. Ninety-nine trials resolve on the first query and span 487–1126 ms with a coefficient of variation of 0.260; one does not, and pays a full retry interval, giving the 3000.8 ms maximum, 1.9 s clear of the next value. That single quantization event, rather than a heavy-tailed process, is what raises the reported coefficient of variation to 0.43 and separates the mean (676.5 ms) from the median (574.1 ms).

Figure 5 compares stage latency distributions on a logarithmic scale. Heartbeat observability is two orders of magnitude faster than enrollment or deployment. Figure 7 shows that deployment contributes most of the tail mass, with p95 1021.8 ms and p99 1145.2 ms against enrollment's. Excluding the single retried trial its dispersion is closer to enrollment's than the headline figures suggest (CV 0.260 against 0.164), the residual difference reflecting that enrollment is timed on the protocol exchange itself whereas deployment is timed on a subsequent status query.

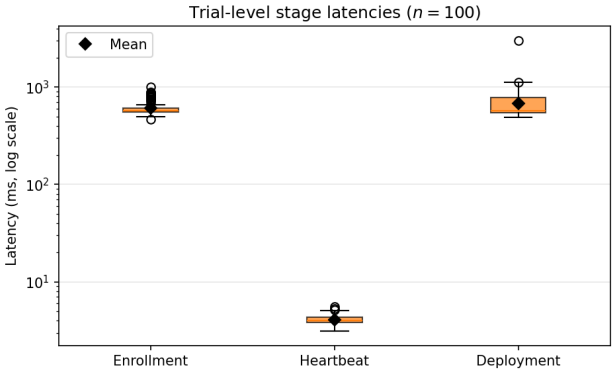

Figure 5: Distribution of trial-level stage latencies (n=100, log scale). Diamonds indicate sample means.

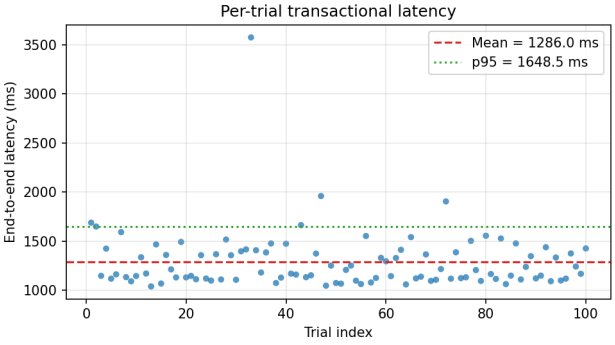

Figure 6: End-to-end latency by trial index (n=100). The series is stationary after the discarded warm-up run; see Section 7 for the correlation statistics.

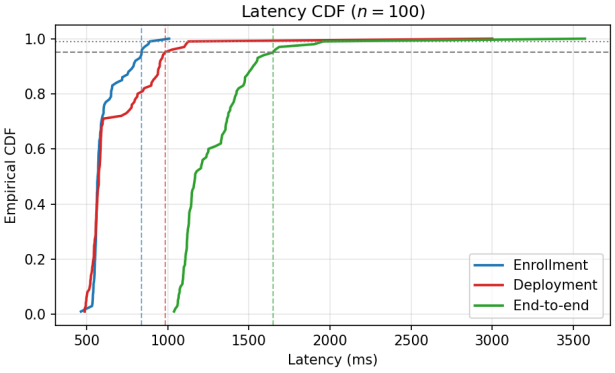

Figure 7: Empirical CDF of enrollment, deployment, and end-to-end latencies (n=100).

7.4. Sustainability-Oriented Metrics

7.4.1. Southbound wire efficiency

Application messages use a 4-byte big-endian length prefix followed by CBOR payloads. Figure 8 summarizes measured wire sizes for representative messages. Heartbeat request and acknowledgment are 6 B each; a complete enrollment round trip totals 87 B of application payload; a minimal deploy round trip (33 B WASM module) totals 90 B. At the

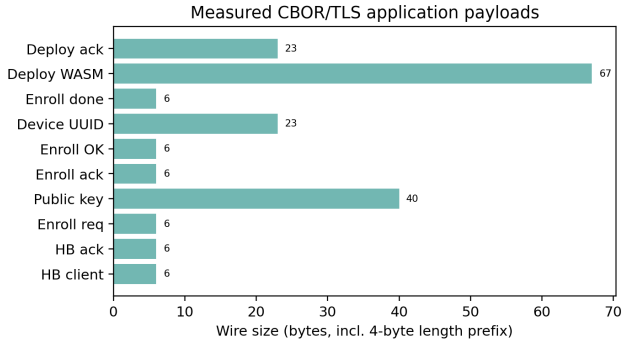

Figure 8: Application-layer wire sizes (4-byte length prefix + CBOR) for representative protocol messages.

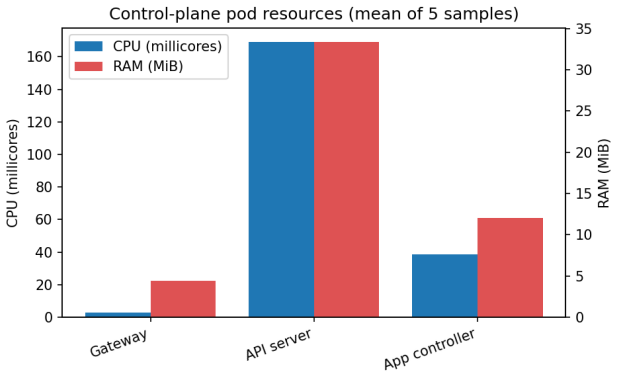

Figure 9: Mean CPU (millicores) and RAM (MiB) for control-plane pods during the sustainability measurement pass.

configured 25 s heartbeat period, steady-state application-layer heartbeat traffic is $12 \times (3600/25) = 1728$ B/h per device (≈1.69 KiB/h), excluding TLS record overhead and lower-layer TCP/IP, Ethernet/TAP, and retransmission overhead.

That exclusion spans more than an order of magnitude. Under the TLS 1.2 AES-128-GCM cipher suite each record adds 5 B of header, an 8 B explicit nonce and a 16 B tag, so a 6 B heartbeat becomes a 35 B record, a 75 B IPv4 segment and a 93 B Ethernet frame. At a 25 s period the delayed-acknowledgment timer expires between exchanges, giving roughly 300 B per exchange and of order 40 KB/h per device, some 25× the application-layer figure. Session establishment is excluded from both and is paid again on every reconnection, so a frequently reconnecting device is dominated by handshake cost rather than by heartbeats. The gateway pins TLS 1.2 to match the cipher suites enabled in the Zephyr mbedTLS build; TLS 1.3 would drop the explicit nonce and save 8 B per record, a change worth making but small against the framing cost itself.

7.4.2. *Edge footprint*

The evaluated Zephyr/WAMR/TLS/CBOR firmware fits the STM32F746NG internal-memory budget: the flash-resident `zephyr.bin` image is 406,568 B (≈397 KiB), corresponding to text plus initialized data, and the static RAM allocation is 270,238 B (≈264 KiB), corresponding to data (9,384 B) plus BSS (260,854 B).[1] The benchmark WASM module remains 33 B. The architectural point supported by these measurements is therefore that embedded endpoints run no kubelet, container runtime, or Linux userspace while retaining an embedded-class memory footprint.

7.4.3. *Concurrent devices*

The trials above repeat one device through the full workflow, which measures the workflow but says nothing about concurrency. A separate pass therefore attaches three emulated STM32F746G devices at the same time. Each device carries its own enrollment key, its own link-layer identity and its own DHCP lease, so the gateway, the control plane and the network segment can all tell them apart; the identity a device presents during enrollment is the one recorded in its Device resource.

All three attach and stay attached. The gateway runtime view lists three distinct sessions, each with a heartbeat fresher than the 25 s firmware emission interval, and the three Device resources independently reach `phase=Connected` with their own liveness evidence. A single Application targeting all three devices converges to `Running` on each of them: the gateway dispatches three separate `DeployApplication` messages, one per session, and receives three successful deployment acknowledgments. Because a deployment command travels only over the session that belongs to its target device, three acknowledged deployments are direct evidence of three independent southbound channels rather than of one channel reused.

Three devices on a single emulation host exercise session multiplexing, per-device identity binding and per-device dispatch, establishing concurrency at the mechanism level; Section 8 states what this does not characterize.

7.4.4. *Control-plane resources*

Figure 9 reports mean Kubernetes pod usage (n=5 samples, one TLS-connected device, sampled while the trial campaign was running). The metric is the container working set reported by `metrics-server`, cross-checked against cgroup v2 `memory.current` and the anonymous resident set in `memory.stat`, which agree with it to within a few hundred kilobytes. The gateway averaged 3.0 millicores CPU and 4.4 MiB RAM (`memory.current` 5.60 MB, anonymous 3.82 MB), the API server averaged 169.2 millicores and 33.4 MiB under campaign load (`memory.current` 32.3–33.0 MB), and the application controller averaged 38.4 millicores and 12.0 MiB (`memory.current` 13.3 MB). Registering 50 synthetic boards through the gateway HTTP registry left gateway resident memory unchanged, bounding the cost of registry state alone: those boards are `Device` entries rather than TLS-connected sessions.

[1] The `zephyr.elf` file is 8,095,176 B (7.72 MiB), but only because it carries ELF metadata and debug symbols for the Renode `sysbus LoadELF` workflow; it is not the image written to device flash.

7.4.5. *Computational energy instrumentation (measured CPU-time; derived energy and carbon as illustrative proxies)*

Beyond control-plane pod resources, we instrument a representative WASM computational workload directly. A Kubernetes pod running the fuel-metered Wasmtime runtime (`wasmbed-wasm-runtime`) repeatedly creates an isolated instance and executes a bounded arithmetic-loop WASM function (summing 0 to N), cycling 30 s idle and 30 s active phases. This instruments a control-plane/fog-layer WASM execution path as a computational-energy proxy. The pod runs under Guaranteed QoS (`requests = limits`, 2 vCPU) to remove self-induced CFS-throttling variance observed at the original 1-vCPU limit (835/27,021 accounting periods throttled under load, a scheduling artifact of the container's own quota, not of co-located tenants).

Two independent signals are collected. First, cgroup CPU time (`cpu.stat usage_usec`) is an exact kernel accounting figure, not an estimate: at N=200,000, across n=39 paired cycles, the pod consumed 29.9289 ± 0.0189 CPU-s during each 30 s active window versus 0.000116 ± 0.000053 CPU-s during each equal-length idle window (paired difference 95% CI [29.9230, 29.9354] s, excluding zero). Second, we deploy Kepler [50] as a DaemonSet and query `kepler_container_joules_total{mode="dynamic"}` scoped to the probe pod's own `pod_name` label, isolating it from co-located wasmbed pods in the same namespace. Because the host is a virtualized guest without exposed RAPL counters, Kepler's output is a regression-based power estimate rather than a direct hardware measurement; with that caveat, it shows a clear and physically plausible increase under load: it attributes no dynamic power to the pod during the idle windows and 3.952 W during the active ones (Figure 10). The exactly-zero idle attribution is a property of the estimator, not a measurement of the pod drawing no power. We therefore treat the exact cgroup CPU-time as the primary, reproducible quantity throughout, and the Kepler wattage as a secondary, model-based signal consistent with it.

Repeating the same idle/active protocol while varying the workload size $N \in \{50{,}000, 200{,}000, 1{,}000{,}000, 5{,}000{,}000\}$ ($n \geq 10$ cycles per size) isolates CPU-time per WASM invocation from the fixed 30 s window: it grows from 95.9 μs at N=50,000 to 9.36 ms at N=5,000,000. The two largest sizes exceed the MCU profile's default Wasmtime fuel budget of 5×10^6 units, which traps the call; they are measured with the budget raised (fuel metering still enabled), a change that alters no other aspect of the workload. Plotted against an $O(N)$ reference line of slope 1 anchored at the smallest measured point, the four measurements track it closely across two orders of magnitude in N, consistent with the loop's linear time complexity rather than merely connecting four samples (Table 9; 95% CIs per point are narrower than the marker at this scale, $n \geq 10$ cycles each).

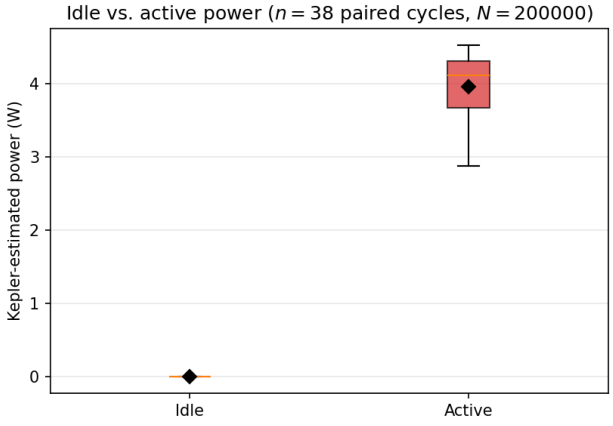

Figure 10: Kepler-estimated power, idle vs. active phase (n=38 paired 30 s cycles with a power reading, N=200,000). Model-based estimate: this host exposes no RAPL counters.

Table 9
Workload size vs. measured CPU-time per invocation. CPU-s/call is exact kernel accounting; J/call is an *illustrative range* spanning the two candidate per-vCPU full-load coefficients (4.72 W measured on the same part by a third party [7], 4.84 W from the manufacturer TDP [1]), and is read as an order of magnitude, not as a measurement.

N	n	CPU-s/call	J/call (illustrative)
50,000	10	9.59×10^{-5}	$4.5\text{–}4.6 \times 10^{-4}$
200,000	39	3.77×10^{-4}	1.8×10^{-3}
1,000,000	10	1.87×10^{-3}	$8.8\text{–}9.0 \times 10^{-3}$
5,000,000	10	9.36×10^{-3}	$4.4\text{–}4.5 \times 10^{-2}$

Converting CPU-seconds into Joules requires a hardware-specific power coefficient, the one step of the pipeline not measured on this platform. The deployed processor is an AMD EPYC 7313P, 16 cores / 32 threads at a 155 W default TDP [1], giving 4.84 W per vCPU; the guest of Table 6 is not oversubscribed, so its 32 vCPUs map one-to-one onto the socket's threads. TDP bounds rather than predicts draw, and a third-party RAPL measurement of the same part reports 4.72 W per vCPU at full load [7], within 3% of it; a platform-specific coefficient would instead come from SPECpower_ssj2008 [49] or the Cloud Carbon Footprint per-family table [15]. The conversion assumes power proportional to allocated vCPU-time at full load, ignoring the idle baseline, DVFS, shared uncore and the non-linearity of power against utilization; each can move the result by tens of percent, so every derived figure is given to at most two significant digits. Applying the Cloud Carbon Footprint average-wattage formula [15] yields ≈119 J for the active window above its idle baseline, ≈1.5 mJ per invocation over ≈79,300 invocations. At a grid intensity I in gCO_2eq/kWh the window accounts for $\approx 33\, I$ μgCO_2eq; the ISPRA consumption-mix factor for Italy, 199.61 gCO_2eq/kWh for 2025 [25], puts it at ≈6.6 $mgCO_2eq$. Production- and consumption-mix factors differ and must be stated with the value [20].

Table 10
CPU-seconds per phase per pod (n=5 trials, board registered through the gateway HTTP endpoint and then enrolled over TLS; all three stages succeeded 5/5). Exact cgroup `cpu.stat`, not an estimate.

Stage	Pod	Mean	Median	Min–max	95% CI (bootstrap)
Enrollment	gateway	0.000350	0.000326	0.000266–0.000503	[0.000291, 0.000432]
Enrollment	api-server	0.143510	0.083265	0.080665–0.384413	[0.082006, 0.264079]
Enrollment	app-controller	0.036625	0.035557	0.030778–0.045817	[0.032261, 0.041684]
Heartbeat	gateway	0.000413	0.000390	0.000227–0.000684	[0.000292, 0.000563]
Heartbeat	api-server	0.020483	0.000000	0.000000–0.102417	[0.000000, 0.061450]
Heartbeat	app-controller	0.010088	0.010080	0.009197–0.010956	[0.009415, 0.010760]
Deployment	gateway	0.006338	0.006521	0.003787–0.008674	[0.004659, 0.008018]
Deployment	api-server	0.564172	0.465255	0.307216–0.823915	[0.390794, 0.739244]
Deployment	app-controller	0.057557	0.027365	0.019152–0.111654	[0.021213, 0.093901]

Per-phase, per-pod CPU-time (control-plane trials). As an instrumentation point layered on the trial harness behind Table 8, leaving gateway, controller and reconciler logic unchanged, each of the enrollment, heartbeat, and deployment stages is separately bracketed by a `cpu.stat usage_usec` snapshot for the gateway, API-server, and application-controller pods, giving CPU-seconds consumed by each pod during exactly that stage's wall-clock window. The measurement runs against the full live stack: a Zephyr 3.5.0/WAMR image for the emulated STM32F746G target, connected to the gateway over TLS through the Renode TAP bridge. Each trial registers the board through the gateway HTTP endpoint, which creates the `Device` resource, and the firmware then drives the same enrollment exchange as the campaign of Table 8, reaching `phase = Connected` with a fresh heartbeat. Registration falls inside the measured window, so these latencies are not comparable with that campaign. Table 10 reports a n=5-trial pass: all three stages succeed 5/5, with real, reproducible CPU-time per stage per pod, and the enrollment row covers the full protocol exchange (TLS handshake, CBOR EnrollmentRequest/PublicKey/DeviceUuid, gateway-side connection established) up to the Device CRD reaching `status.phase = Connected`.

Given n=5, this table demonstrates the instrumentation working end-to-end on real infrastructure rather than providing a statistically powered per-stage estimate; a n=100 run aligned with Table 8 is future work.

7.4.6. Architectural comparison

Table 11 contrasts gateway-mediated WASM orchestration with treating each constrained device as a Kubernetes worker. Edge Kubernetes deployments require a Linux-capable node with a kubelet, container runtime, and supporting operating-system services before user workloads, whereas the evaluated endpoint keeps those cluster agents off the device. Our design concentrates orchestration in a shared gateway (under 5 MiB measured) while keeping cluster agents off the device. Only the right-hand column is instrumented here; establishing the left-hand one would require the same WASM workload on a Linux single-board node with a cluster agent. The pod measurements also locate the cost differently: the gateway averages 3.0 millicores against the API server's 169.2, so the control plane rather than the fog layer dominates at this scale, and the fleet size at which gateway mediation overtakes per-device agents is not determined here.

Table 11
Qualitative contrast of orchestration models. Only the gateway-mediated column is measured in this work.

Aspect	Device as K8s worker	Gateway-mediated WASM edge
Edge runtime	kubelet + container runtime + OS	Zephyr + WAMR (397 KiB flash, 264 KiB RAM)
Edge cluster agent footprint	kubelet + container runtime + OS services required	none on device
Steady heartbeat traffic	CRI/container probes	1728 B/h per device (application layer), ≈40 KB/h on the wire
Fog mediation RAM	per-node agent	<5 MiB shared gateway

8. Discussion

8.1. Architectural and RI Implications

The main architectural lesson is that the gateway should be treated as a semantic boundary, not as a transparent relay. Constrained devices can remain focused on protected communication, local execution, and evidence emission, while the gateway handles identity validation, protocol translation, and status propagation toward Kubernetes. This separation avoids pushing cloud-facing semantics into firmware and gives the control plane a stable interface even if the southbound protocol evolves.

The evaluation confirms that protocol success alone is not sufficient: enrollment, liveness, and deployment evidence must be interpreted consistently before they become trustworthy control-plane state. This supports the central design decision to separate desired state, gateway-derived evidence, and controller reconciliation. Future extensions such as multi-gateway placement, staged rollout, or retry policies should preserve this separation rather than allowing multiple components to write conflicting interpretations of device status.

For sustainable digital research infrastructures, the same separation creates explicit attachment points for energy and carbon policies. Placement decisions can be expressed as desired state, gateway and device observations can be treated as evidence, and sustainability controllers can consume the same status model without requiring constrained devices to run additional cloud-native agents. The measured baselines in Section 7.4, byte-scale southbound traffic, single-digit-MiB gateway mediation, and no kubelet on the edge, suggest that subsequent green-experimentation studies can focus on policy and instrumentation rather than on redesigning the orchestration substrate.

The practical implication for research infrastructures is that orchestration evidence becomes a reusable experimental

object. A placement policy, for example, can be evaluated by inspecting which Application resources were scheduled to which devices, which gateway mediated the execution, whether heartbeat evidence remained fresh, and whether the reported execution outcome matched the intended state. The same record can later be joined with external measurements such as site power, carbon-intensity annotations, thermal constraints, or gateway utilization. The current prototype does not optimize those signals, but it provides the control-plane hooks that a sustainability-aware controller would need in order to make such optimization auditable.

8.2. Validity and Scope

The validation targets system-level orchestration behavior rather than real-time performance, radio behavior, or sustainability optimization. Emulation and scripted trials are appropriate for exercising enrollment, deployment, status convergence, and reconciliation semantics, while physical boards, wireless links, and power meters require additional characterization. The results should therefore be interpreted as evidence that the orchestration model is coherent under the evaluated workflow.

External validity is bounded by the single-node K3s deployment, one gateway path, and a minimal WASM workload selected to isolate orchestration overhead rather than application-computation cost. Larger deployments introduce gateway contention, distributed failure modes, network partitions, and device mobility. Concurrency is exercised rather than assumed: three emulated devices hold three distinct mutually authenticated TLS sessions and each receives its own deployment (Section 7.4.3). That evidence stops at three devices on one host, and the 50-board exercise scales registry state rather than sessions, so neither extends to multiple gateways or to fleet-scale energy characterization. Construct validity is also scoped: the study measures identity-bound attachment, observable liveness, deployment completion, latency, wire sizes, firmware footprint, and control-plane RAM/CPU samples, while direct, hardware-validated energy consumption and long-duration reliability remain outside the current evaluation; Section 7.4.5 is a step toward the latter, with exact cgroup CPU-time as the primary signal.

Scope of the energy signal. cgroup CPU-time (`cpu.stat`) is the only quantity in Section 7.4.5 that is an exact, reproducible measurement rather than a model output, and it alone should be treated as load-bearing for any energy claim in this paper. A hardware-validated absolute wattage for this workload, on a host that combines a working k3s control plane with exposed RAPL counters, remains future work.

Conclusion validity is strengthened by the transactional success criterion: enrollment, heartbeat visibility, and deployment must succeed within the same trial. This avoids overstating correctness from isolated protocol stages. Broader claims about optimality, scalability, or comparative advantage require additional experiments against alternative orchestration designs and physical-device deployments.

8.3. Scalability and Failure Modes

The main scalability pressure point is the gateway, because it concentrates southbound sessions, protocol parsing, and status propagation. This is an intentional trade-off: moving the boundary into the gateway avoids embedding Kubernetes behavior in every device, but it also means that future deployments must treat gateways as schedulable and observable resources. Multi-gateway operation should therefore add explicit placement policies, per-gateway capacity metadata, and failover rules rather than relying on implicit network reachability. In such a design, a Device resource would declare or discover a preferred gateway, while Application reconciliation would resolve the current gateway association before attempting deployment.

Four fault categories require distinct recovery actions and are therefore kept separate: transport faults (TLS interruption, reboot, heartbeat loss) call for reconnection; protocol faults (malformed CBOR, identity mismatch, stale sessions, unassociable deployment results) for rejection; control-plane faults (delayed patches, controller restarts, conflicting reconcilers) for preserving desired state; and application faults (WASM load failure, runtime error, policy violation) for surfacing an explicit error.

These considerations also limit the interpretation of the present results. The 100-trial campaign shows that the nominal enrollment-to-deployment transaction is coherent under the evaluated emulated conditions. It does not establish maximum gateway capacity, behavior under network partitions, real-time scheduling guarantees, or recovery time after simultaneous failures. Those are natural next experiments once the single-device protocol and reconciliation invariants are fixed.

8.4. Reproducibility

The evaluation is designed to be repeatable on a commodity Linux host with Docker and K3s. Renode makes the embedded side deterministic enough for regression-style validation, although it remains complementary to future physical-board experiments. Evidence traceability is maintained across multiple layers: gateway logs, API-visible resource states, and Kubernetes CRD status describe the same workflow from different perspectives, so that protocol success and control-plane convergence are not inferred from a single eventually consistent field.

For a replication package, the most important artifacts are therefore not only the source code and container images, but also the CRD manifests, firmware configuration, WASM module, trial harness, raw latency records, and the scripts that generate the figures. Reporting these artifacts together would allow another researcher to distinguish implementation regressions from environmental variation. A failed replication could then be localized to a specific layer: Kubernetes resource creation, gateway mediation, TLS/CBOR communication, embedded execution, or post-processing of the measurement records. If institutional constraints prevent publishing all raw records, the artifact statement should

separate public assets, such as source code, manifests, and scripts, from records available on request.

9. Conclusion and Future Work

This paper presented a Kubernetes-native framework for managing embedded WebAssembly workloads across the Cloud–Fog–Edge continuum without requiring constrained devices to become Kubernetes nodes. The contribution integrates CRD-based intent, gateway-mediated TLS/CBOR communication, Zephyr firmware, WAMR execution, and evidence-driven reconciliation into a single verifiable operational path suitable for sustainable digital research infrastructures.

The experimental evaluation on a K3s testbed with emulated embedded targets confirms that the architecture supports a complete enrollment-to-deployment workflow under repeatable conditions, with 100/100 transactional success and approximately 1.29 s mean end-to-end orchestration latency, a figure that includes the granularity with which the harness observes deployment status. The measured application-layer traffic, firmware footprint, and fog-layer resource use provide quantitative anchors for subsequent energy-aware placement, carbon-aware annotation, and green-experimentation studies, while direct energy optimization remains future work. The computational-energy pass of Section 7.4.5 has the same scope: it instruments a Wasmtime runtime at the fog layer rather than the Zephyr/WAMR firmware on the embedded target, and its Joule and carbon figures are proxies derived from measured CPU-time with an external power coefficient.

Future work will extend the platform toward multi-gateway deployments, fault injection, physical hardware, wattmeter- or RAPL-validated energy measurements, carbon-intensity annotation beyond the illustrative, derived-estimate pass reported in Section 7.4.5, and larger device populations.

Data Availability Statement

The data supporting the findings of this study consist of deployment logs, Kubernetes resource states, controller and gateway traces, and protocol evidence from controlled experiments. The source repository provides the public deployment assets needed to inspect the framework. The measurement artifacts underlying every number reported in Section 7 — the per-trial latency and success records of the n=100 campaign, the paired idle/active and workload-size energy records, the per-phase `cpu.stat` records, the pod-resource samples, and the analysis and figure-generation scripts that turn them into the tables and figures of this paper — are available in the source repository below. Records not included in the repository are limited to raw operational logs that carry institutional infrastructure identifiers; these are available from the corresponding author upon reasonable request, subject to institutional policies and applicable confidentiality constraints.

Code Availability Statement

The source code and deployment assets associated with the framework are maintained in the public project repository `github.com/fabiooraziomirto/k3s-wasm-edge-orchestration`. The exact revision evaluated in this paper is tagged `v1.0`, so that the results can be reproduced from a fixed snapshot rather than from a moving branch.

Funding

This research received no external funding.

Conflict of Interest

The authors declare that they have no conflict of interest related to this work.

Author Contributions

Conceptualization: Giovanni Merlino, with input from Francesco Longo. Methodology: Luca D'Agati, Giuseppe Tricomi, Fabio Orazio Mirto, and Giovanni Merlino. Software: Luca D'Agati, with contributions from Giuseppe Tricomi and Fabio Orazio Mirto. Validation: Luca D'Agati, Giuseppe Tricomi, and Fabio Orazio Mirto. Investigation and data curation: Luca D'Agati, Giuseppe Tricomi, and Fabio Orazio Mirto. Writing—original draft: Luca D'Agati, with writing support from Giuseppe Tricomi and Fabio Orazio Mirto. Writing—review and editing: Luca D'Agati, Giuseppe Tricomi and Fabio Orazio Mirto. Supervision: Giovanni Merlino, with support from Francesco Longo.

References

[1] Advanced Micro Devices, 2021. Amd epyc 7313p processor specifications. https://www.amd.com/en/products/processors/server/epyc/7003-series/amd-epyc-7313p.html. 16 cores / 32 threads; default TDP 155 W, configurable TDP 155–180 W; Accessed: 2026-08-19.

[2] Akri Authors, 2026. Akri: A kubernetes resource interface for the edge. https://docs.akri.sh/. Accessed: 2026-05-15.

[3] Amazon Web Services, 2026a. Aws iot greengrass documentation. https://docs.aws.amazon.com/greengrass/. Accessed: 2026-05-15.

[4] Amazon Web Services, 2026b. Freertos documentation. https://www.freertos.org/. Accessed: 2026-05-15.

[5] Antmicro, 2026. Renode: Development framework for multi-node embedded networks. https://renode.io/. Accessed: 2026-05-15.

[6] Baccelli, E., Hahm, O., Gunes, M., Wahlisch, M., Schmidt, T.C., 2013. RIOT OS: Towards an os for the internet of things, in: Proceedings of the IEEE Conference on Computer Communications Workshops, pp. 79–80.

[7] Balci, M., 2023. Amd epyc 7313p energy consumption test. https://metebalci.com/blog/epyc-energy-consumption-test/. Third-party RAPL/turbostat PkgWatt measurement, single-socket, 32 threads, used here only as secondary empirical corroboration; Accessed: 2026-07-29.

[8] Bonomi, F., Milito, R., Zhu, J., Addepalli, S., 2012. Fog computing and its role in the internet of things, in: Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing, pp. 13–16.

[9] Bormann, C., Hoffman, P., 2020. RFC 8949: Concise binary object representation (cbor). RFC 8949.

[10] Bytecode Alliance, 2026a. Wasmtime: A fast and secure runtime for webassembly. https://wasmtime.dev/. Accessed: 2026-05-15.

[11] Bytecode Alliance, 2026b. Webassembly system interface (wasi). https://wasi.dev/. Accessed: 2026-05-15.

[12] Bytecode Alliance and WAMR Contributors, 2026. Webassembly micro runtime (wamr). https://github.com/bytecodealliance/wasm-micro-runtime. Accessed: 2026-05-15.

[13] Canonical, 2026. Microk8s documentation. https://microk8s.io/docs. Accessed: 2026-05-15.

[14] Chiang, M., Zhang, T., 2016. Fog and iot: An overview of research opportunities. IEEE Internet of Things Journal 3, 854–864.

[15] Cloud Carbon Footprint, 2026. Cloud carbon footprint: Methodology. https://www.cloudcarbonfootprint.org/docs/methodology/. Accessed: 2026-07-29.

[16] containerd Contributors, 2026. runwasi: Webassembly runtime integration for containerd. https://github.com/containerd/runwasi. Accessed: 2026-05-15.

[17] D'Agati, L., Tricomi, G., Arena, M., Longo, F., Puliafito, A., Merlino, G., 2025. Iot orchestration in the compute continuum: Integrating kubernetes with stack4things, in: 2025 IEEE 25th International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW), pp. 140–147. doi:10.1109/CCGridW65158.2025.00028.

[18] Dunkels, A., Gronvall, B., Voigt, T., 2004. Contiki – a lightweight and flexible operating system for tiny networked sensors, in: Proceedings of the 29th Annual IEEE International Conference on Local Computer Networks, pp. 455–462.

[19] EdgeX Foundry, 2026. Edgex foundry documentation. https://www.edgexfoundry.org/. Accessed: 2026-05-15.

[20] European Environment Agency, 2025. Greenhouse gas emission intensity of electricity generation in europe. https://www.eea.europa.eu/en/analysis/indicators/greenhouse-gas-emission-intensity-of-1. Country-level indicator; Accessed: 2026-08-19.

[21] Fdida, S., Makris, N., Korakis, T., Bruno, R., Passarella, A., Andreou, P., Belter, B., Crettaz, C., Dabbous, W., Demchenko, Y., et al., 2022. Slices, a scientific instrument for the networking community. Computer Communications 193, 189–203.

[22] Fermyon Technologies, 2026. Spin: Framework for webassembly applications. https://www.fermyon.com/spin. Accessed: 2026-05-15.

[23] FIWARE Foundation, 2026. Fiware documentation. https://www.fiware.org/. Accessed: 2026-05-15.

[24] Haas, A., Rossberg, A., et al., 2017. Bringing the web up to speed with webassembly, in: Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), pp. 185–200.

[25] ISPRA – Istituto Superiore per la Protezione e la Ricerca Ambientale, 2026. Settore elettrico: emissioni di CO2 e altri impatti. Edizione 2026. Rapporti 430/2026. ISPRA. URL: https://www.isprambiente.gov.it/it/pubblicazioni/rapporti/settore-elettrico-emissioni-di-co2-e-altri-impatti-edizione-2026. national emission factors for electricity production and consumption.

[26] Krustlet Authors, 2026. Krustlet: Kubernetes kubelet in rust for webassembly workloads. https://krustlet.dev/. Accessed: 2026-05-15.

[27] KubeEdge Authors, 2026. Kubeedge documentation. https://kubeedge.io/. Accessed: 2026-05-15.

[28] Kubernetes Community, 2026. Operator pattern. https://kubernetes.io/docs/concepts/extend-kubernetes/operator/. Accessed: 2026-05-15.

[29] Levis, P., Madden, S., Polastre, J., Szewczyk, R., Whitehouse, K., Woo, A., Gay, D., Hill, J., Welsh, M., Brewer, E., Culler, D., 2005. Tinyos: An operating system for sensor networks. Ambient Intelligence , 115–148.

[30] Microsoft, 2026. Azure iot edge documentation. https://learn.microsoft.com/azure/iot-edge/. Accessed: 2026-05-15.

[31] Mirto, F.O., D'Agati, L., Tricomi, G., Longo, F., Puliafito, C., Virdis, A., Merlino, G., 2026. Toward deviceless paradigm: An iot-centered and orchestrated FaaS architecture, in: 2026 IEEE International Conference on Smart Computing Workshops and Other Affiliated events (SmartComp Companion), pp. 243–248. doi:10.1109/SmartComp-Companion70724.2026.00058.

[32] OASIS, 2019. MQTT version 5.0. OASIS Standard.

[33] oneM2M, 2026. onem2m technical specifications. https://www.onem2m.org/technical/published-specifications. Accessed: 2026-05-15.

[34] OPC Foundation, 2026. Opc unified architecture specification. https://opcfoundation.org/about/opc-technologies/opc-ua/. Accessed: 2026-05-15.

[35] Open Mobile Alliance, 2022. Lightweight machine to machine technical specification. OMA SpecWorks.

[36] OpenYurt Authors, 2026. Openyurt: Extending kubernetes to edge computing. https://openyurt.io/. Accessed: 2026-05-15.

[37] Osterlind, F., Dunkels, A., Eriksson, J., Finne, N., Voigt, T., 2006. Cross-level sensor network simulation with cooja, in: Proceedings of the 31st IEEE Conference on Local Computer Networks, pp. 641–648.

[38] Rancher Labs, 2026. K3s: Lightweight kubernetes. https://k3s.io/. Accessed: 2026-05-15.

[39] Rescorla, E., 2018. RFC 8446: The transport layer security (tls) protocol version 1.3. RFC 8446.

[40] Rescorla, E., Tschofenig, H., Modadugu, N., 2022. RFC 9147: The datagram transport layer security (dtls) protocol version 1.3. RFC 9147.

[41] Richardson, M., van der Stok, P., Kampanakis, P., 2021. RFC 9019: A firmware update architecture for internet of things. RFC 9019.

[42] Riley, G.F., Henderson, T.R., 2010. The ns-3 network simulator, in: Modeling and Tools for Network Simulation, Springer. pp. 15–34.

[43] Satyanarayanan, M., 2017. The emergence of edge computing. Computer 50, 30–39.

[44] Schaad, J., 2022. RFC 9052: Cbor object signing and encryption (cose): Structures and process. RFC 9052.

[45] Selander, G., Mattsson, J., Palombini, F., Seitz, L., 2019. RFC 8613: Object security for constrained restful environments (oscore). RFC 8613.

[46] Shelby, Z., Hartke, K., Bormann, C., 2014. RFC 7252: The constrained application protocol (coap). RFC 7252.

[47] Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L., 2016. Edge computing: Vision and challenges. IEEE Internet of Things Journal 3, 637–646.

[48] SPIFFE Authors, 2026. SPIFFE: Secure production identity framework for everyone. https://spiffe.io/. Accessed: 2026-05-15.

[49] Standard Performance Evaluation Corporation, 2008. SPECpower_ssj2008. https://www.spec.org/power_ssj2008/. Server power and performance benchmark; per-platform published results; Accessed: 2026-08-19.

[50] Sustainable Computing IO, 2026. Kepler: Kubernetes-based efficient power level exporter. https://sustainable-computing.io/. Accessed: 2026-07-29.

[51] The Kubernetes Authors, 2026. Kubernetes documentation. https://kubernetes.io/docs/. Accessed: 2026-05-15.

[52] The Update Framework Authors, 2026. The update framework specification. https://theupdateframework.io/. Accessed: 2026-05-15.

[53] The Zephyr Project, 2026. Zephyr project documentation. https://docs.zephyrproject.org/. Accessed: 2026-05-15.

[54] Varga, A., Hornig, R., 2008. An overview of the omnet++ simulation environment. Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems .

[55] Wasm3 Authors, 2026. Wasm3: A fast webassembly interpreter. https://github.com/wasm3/wasm3. Accessed: 2026-05-15.

[56] WasmEdge Authors, 2026. Wasmedge runtime. https://wasmedge.org/. Accessed: 2026-05-15.

[57] World Wide Web Consortium, 2019. Webassembly core specification. W3C Recommendation.